

Symfony

Introduction

Framework PHP: définition

Un framework:

- offre un cadre de travail
- propose une bibliothèque de fonctionnalités dans laquelle les développeurs vont pouvoir piocher en fonction de leurs besoins.
- permet donc de gagner du temps
- est comparable à une boîte à outils

Un framework est construit autour d'un langage de programmation : PHP, Python, javascript, Ruby, etc

Framework PHP: revue du marché

Back:

- Symfony, Laravel, Zend (PHP)
- Django, Pyramid (Python)
- Ruby on Rails (Ruby)
- Struts, Hibernate, Spring (JAVA)

Framework PHP: revue du marché

Back:

- Symfony, Laravel, Zend (PHP)
 - Django, Pyramid (Python)
 - Ruby on Rails (Ruby)
 - Struts, Hibernate, Spring (JAVA)
-

Front: (JavaScript)

- Angular
- VueJS
- React

<https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web/#:~:text=js%20overtook%20React.,40.6%20percent%20were%20using%20React.>

Principes d'architecture

Centrage HTTP

Symfony est étroitement calqué sur le flux HTTP Request-Response : ce paradigme fondamental qui est derrière presque toutes les communications sur le web.

Orientation objet

L'ensemble du framework utilise une syntaxe orienté objet pour construire l'application.

Cela signifie que si le choix se présente, il faudra systématiquement choisir une version orienté objet d'une fonctionnalité de PHP

Ex: les dates

Principes d'architecture

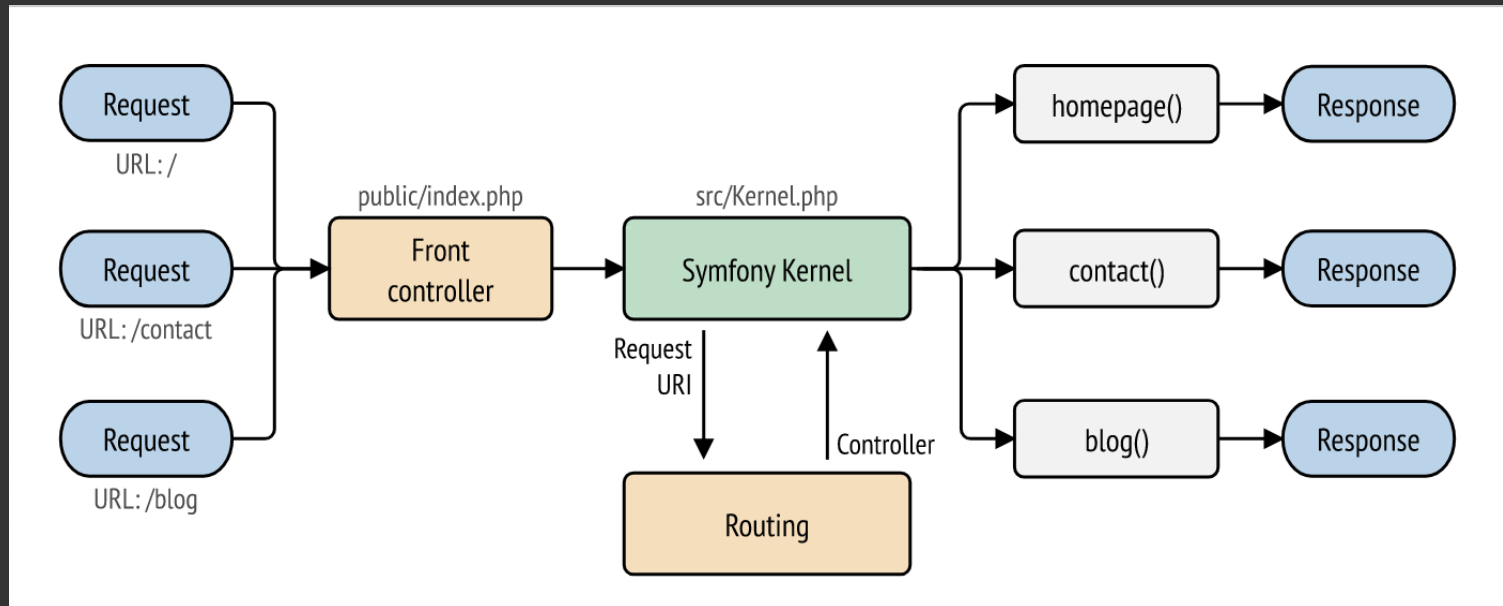
Orientation service

Dans Symfony, les objets sont appelés services et chaque service vit à l'intérieur d'un objet très spécial appelé le conteneur de services. Le conteneur permet de centraliser la manière dont les objets sont construits. Il vous facilite la vie, favorise une architecture solide et est ultra rapide !

Depuis la version 4 de Symfony, presque toutes les classes d'un projet sont des services.

Principes d'architecture

Le flux d'application Symfony



Anatomie du framework

le kernel

Le **Kernel** est le coeur de Symfony. Il se charge de mettre en place tous les bundles utilisés par l'application et de leur fournir la configuration de l'application. Il crée ensuite le conteneur de service avant de servir les requêtes.

les composants

Les **composants** Symfony sont des bibliothèques PHP autonomes fournissant une fonctionnalité spécifique, qui peut être utilisée dans n'importe quelle application PHP.

les bundles

Un **bundle** est similaire à un plugin dans d'autres logiciels, mais en mieux. Les principales fonctionnalités du framework Symfony sont implémentées avec des bundles (FrameworkBundle, SecurityBundle, DebugBundle, etc.) Ils sont également utilisés pour ajouter de nouvelles fonctionnalités dans votre application.

Les **bundles** ne doivent être utilisés que pour partager du code et des fonctionnalités entre plusieurs applications.

Installer et configurer un projet

Configurer l'environnement

Les pré-requis

Avant de créer votre première application Symfony vous devez :

- Installez PHP 8.1 ou supérieur et ces extensions PHP (qui sont installées et activées par défaut dans la plupart des installations PHP 8) : Ctype, iconv, PCRE, Session, SimpleXML et Tokenizer ;
- Installez [Composer](#), qui est utilisé pour installer les packages PHP.

L'exécutable symfony

Vous pouvez également installer [Symfony CLI](#). Cela crée un binaire appelé symfony qui fournit tous les outils dont vous avez besoin pour développer et exécuter votre application Symfony localement.

Symfony CLI fournit également un outil pour vérifier si votre ordinateur répond à toutes les exigences.

Dans votre terminal, exécutez cette commande : `symfony check:requirements`

Démarrer le projet avec symfony ou composer, choisir son squelette de départ

Avec Symfony CLI

- Pour construire une application web:

```
symfony new my_project_directory --webapp
```

- Pour construire une application microservice, console, API:

```
symfony new my_project_directory
```

On peut aussi ajouter l'option `--version="6.3.*"` pour choisir la version à utiliser

L'option `--webapp` installe tous les packages dont vous avez généralement besoin pour créer des applications Web.

Démarrer le projet avec symfony ou composer, choisir son squelette de départ

Avec Composer

- Pour construire une application web:

```
composer create-project symfony/skeleton my_project_directory
cd my_project_directory
composer require webapp
```

- Pour construire une application microservice, console, API:

```
composer create-project symfony/skeleton my_project_directory
```

On peut aussi ajouter l'option `symfony/skeleton:"6.3.*"` pour choisir la version à utiliser

Ajouter des paquets avec Symfony Flex pour composer

Symfony Flex est un plugin Composer. Il se branche sur les événements Composer dès lors que vous lancez une commande qui installe, met à jour ou supprime un paquet PHP ou bundle Symfony.

Son but est d'automatiser l'installation et la suppression de vos dépendances en fournissant une configuration par défaut sans avoir à aller lire la doc pour trouver quelle configuration écrire, quelles routes charger ou autre tâche à effectuer.

Ajouter des paquets avec Symfony Flex pour composer

Pour installer un composant, on utilise la commande suivante:

```
composer require package/component-name-or-alias
```

Ajouter des paquets avec Symfony Flex pour composer

Pour installer un composant, on utilise la commande suivante:

```
composer require package/component-name-or-alias
```

Deux choses se passent grâce à Symfony Flex:

Ajouter des paquets avec Symfony Flex pour composer

Pour installer un composant, on utilise la commande suivante:

```
composer require package/component-name-or-alias
```

Deux choses se passent grâce à Symfony Flex:

- Quand on utilise un alias Flex pointe vers le vrai nom du package, (ex: l'alias **twig** pointe **symfony/twig-bundle**) et se charge de résoudre cet alias pour Composer.

Ajouter des paquets avec Symfony Flex pour composer

Pour installer un composant, on utilise la commande suivante:

```
composer require package/component-name-or-alias
```

Deux choses se passent grâce à Symfony Flex:

- Quand on utilise un alias Flex pointe vers le vrai nom du package, (ex: l'alias **twig** pointe **symfony/twig-bundle**) et se charge de résoudre cet alias pour Composer.
- Flex installe une recette pour package. Une recette est un moyen pour une bibliothèque de se configurer automatiquement en ajoutant et en modifiant des fichiers. Grâce aux recettes, l'ajout de fonctionnalités est transparent et automatisé.

Liste des recettes : [RECIPES.md](#)

Ajouter des paquets avec Symfony Flex pour composer

Pour installer un composant, on utilise la commande suivante:

```
composer require package/component-name-or-alias
```

Deux choses se passent grâce à Symfony Flex:

- Quand on utilise un alias Flex pointe vers le vrai nom du package, (ex: l'alias **twig** pointe **symfony/twig-bundle**) et se charge de résoudre cet alias pour Composer.
- Flex installe une recette pour package. Une recette est un moyen pour une bibliothèque de se configurer automatiquement en ajoutant et en modifiant des fichiers. Grâce aux recettes, l'ajout de fonctionnalités est transparent et automatisé.

Liste des recettes : [RECIPES.md](#)

Pour supprimer un composant, on utilise la commande suivante:

```
composer remove package/component-name-or-alias
```

Les répertoires du projet

structure des répertoires

- bin/
- composer.json
- composer.lock
- config/
- public/
- src/
- symfony.lock
- var/
- vendor/

Les répertoires du projet

structure des répertoires

Le répertoire `bin/` contient le principal point d'entrée de la ligne de commande : console.

Le répertoire `config/` est constitué d'un ensemble de fichiers de configuration sensibles, initialisés avec des valeurs par défaut.

Le répertoire `public/` est le répertoire racine du site web, et le script `index.php` est le point d'entrée principal de toutes les ressources HTTP dynamiques. Tout ce qui se trouve dans le répertoire `public/` est accessible par un navigateur.

Si on place un fichier image dans un répertoire `public/images/`, il sera alors disponible à l'adresse <http://127.0.0.1:8000/images/mon-image.jpg>.

Le répertoire `src/` héberge tout le code que vous allez écrire ; c'est ici que vous passerez la plupart de votre temps.

Le répertoire `var/` contient les caches, les logs et les fichiers générés par l'application lors de son exécution.

Le répertoire `vendor/` contient tous les paquets installés par Composer, y compris Symfony lui-même.

Configurer le projet via les fichiers de config

Les applications Symfony sont configurées avec les fichiers stockés dans le répertoire `config/`

```
|— config/
|   |— packages/
|   |— bundles.php
|   |— routes.yaml
|   |— services.yaml
```

- `routes.yaml` définit la configuration de routage
- `services.yaml` configure les services du conteneur de services
- `bundles.php` active/désactive les packages dans votre application
- `config/packages/` stocke la configuration de chaque package installé dans votre application.

Configurer le projet via les fichiers de config

Symfony n'impose pas de format spécifique pour configurer vos applications, mais laisse le choix entre YAML, XML et PHP.

Il n'y a pas de différence pratique entre les formats. Symfony les transforme tous en PHP et les met en cache avant d'exécuter l'application, il n'y a donc même pas de différence de performances.

YAML est utilisé par défaut lors de l'installation de packages car il est concis et très lisible.

Définir un paramètre

```
# config/services.yaml
parameters:
    app.foo: 'bar'
```

Paramétrer son environnement via DotEnv

Les applications Symfony sont créées avec un fichier appelé `.env` situé dans le répertoire racine du projet. Ce fichier est utilisé pour définir la valeur des variables d'environnement. (`APP_ENV`, `APP_DEBUG`, `DATABASE_URL`, `MAILER_DSN`, etc)

L'utilisation de variables d'environnement est une pratique courante pour configurer des options qui dépendent de l'endroit où l'application est exécutée (par exemple, les informations d'identification de la base de données sont généralement différentes en production par rapport à votre machine locale).

`/!\` Ce fichier peut être versionné dans votre dépôt et donc ne doit contenir que des valeurs "par défaut". Ce fichier ne doit pas contenir de valeurs de production.

Remplacer les valeurs d'environnement via `.env.local`

Si vous devez remplacer une valeur d'environnement (par exemple, une valeur différente sur votre machine locale), vous pouvez le faire dans un fichier `.env.local`.

`#!/` Ce fichier doit être ignoré par git et ne doit pas être versionné dans votre dépôt.

Atelier: Installer, configurer et lancer un projet Symfony

Créer des pages: les contrôleurs

Générer des contrôleurs

Le composant **Maker** de Symfony.

Maker aide à créer des commandes vides, des contrôleurs, des classes de formulaires, des tests, etc

Pour créer un contrôleur à l'aide de **Maker**, on utilise la commande suivante

```
php bin/console make:controller controller-name
```

La commande va générer 2 fichiers:

- src/Controller/ControllerNameController.php
- templates/controller-name/index.html.twig

(!) Voir la liste des commandes disponibles: `php bin/console`

Les contrôleurs

Un contrôleur:

- est une fonction PHP
- qui lit les informations de l'objet Request
- crée et renvoie un objet `Response`

Les contrôleurs héritent de la classe `AbstractController` et doivent retourner une instance de la classe `Response`. Par défaut, les contrôleurs sont définis avec l'attribut php `Route`, pour y associer une url.

La réponse peut être une page HTML, JSON, XML, un téléchargement de fichier, une redirection, une erreur 404 ou autre.

Les contrôleurs

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class SandboxController extends AbstractController
{
    #[Route('/sandbox', name: 'app_sandbox')]
    public function index(): Response
    {
        return $this->render('sandbox/index.html.twig', [
            'controller_name' => 'SandboxController',
        ]);
    }
}
```

Les contrôleurs

Pour afficher le résultat d'un contrôleur, vous devez utiliser l'URL défini dans la classe `Route`.

Si on utilise `#[Route('/my-controller-url')]`, alors pour voir le résultat de cette page, on y accède avec cette URL : <http://127.0.0.1:8000/my-controller-url>

La classe `Route` permet également de définir un paramètre "name", pour associer un nom à une route. Ce nom est utilisé pour générer les urls à l'aide du service `UrlGenerator`.

Les contrôleurs

Générer des URLs

La méthode `generateUrl()` fournie par la classe `AbstractController`, permet de générer un chemin absolue à partir du nom d'une route.

```
$url = $this->generateUrl('route_name');
```

Pour créer une URL absolue, on peut fournir la constante

`UrlGeneratorInterface::ABSOLUTE_URL` au paramètre `referenceType`

```
$urlAbsolue = $this->generateUrl("route_name", referenceType: UrlGeneratorInterface::ABSOLUTE_URL);
```

Les contrôleurs

Les erreurs HTTP

404 Not found

```
throw $this->createNotFoundException();
```

401 Unauthorized

```
throw $this->createAccessDeniedException();
```

500 Internal server error

```
throw new \Exception();
```

Les contrôleurs

Les Réponses

La classe `Response` permet de fournir une réponse HTTP utilisant une interface orientée objet pour construire la réponse qui doit être renvoyée.

Il existe plusieurs classe qui héritent de la classe `Response`

- `JSONResponse` qui fournit une réponse au format json
- `RedirectResponse` qui permet de rediriger vers une autre URL
- `BinaryFileResponse` qui permet le téléchargement de document
- `StreamedResponse` qui permet de diffuser une réponse représenté par une fonction PHP

Les contrôleurs

Réponse HTTP: Response

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ResponseController extends AbstractController
{
    #[Route('/response', name: 'response_index')]
    public function index(): Response
    {
        return new Response('[...]');

        ou

        return $this->render('response/index.html.twig', [
            'controller_name' => 'ResponseController',
        ]);
    }
}
```

Les contrôleurs

Json: JsonResponse

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

class ResponseController extends AbstractController
{
    #[Route('/response/json', name: 'response_json')]
    public function jsonResponse(): JsonResponse
    {
        $user = ['firstname' => "John", 'lastname' => "Doe", 'email' => "john.doe@gmail.com"];

        return $this->json($user);

        ou

        return new JsonResponse($user);
    }
}
```


Les contrôleurs

Téléchargement: BinaryFileResponse

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\BinaryFileResponse;
use Symfony\Component\Routing\Annotation\Route;

class ResponseController extends AbstractController
{
    #[Route('/response/file', name: 'response_file')]
    public function fileResponse(): BinaryFileResponse
    {

        return $this->file('download/cafe.jpg');

        ou

        return new BinaryFileResponse('download/cafe.jpg');
    }
}
```

Les contrôleurs

Redirection: RedirectResponse

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\RedirectResponse;
use Symfony\Component\Routing\Annotation\Route;

class ResponseController extends AbstractController
{
    #[Route('/response/redirection', name: 'response_redirection')]
    public function redirection(): RedirectResponse
    {
        return $this->redirect($this->generateUrl('route_name'));

        ou

        return $this->redirectToRoute('route_name');

        ou

        return new RedirectResponse($this->generateUrl('route_name'));

        ou

        return $this->redirect('http://www.dawan.fr');
    }
}
```

Les contrôleurs

L'objet Request

Si vous avez besoin de lire les paramètres d'une requête, de récupérer un en-tête de requête ou d'accéder à un fichier téléchargé, ces informations sont stockées dans l'objet `Request` (Package `HttpFoundation`).

Symfony transmettra l'objet `Request` à n'importe quel contrôleur qui le définit en argument.

```
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;

public function index(Request $request): Response
{
    [...]
}
```

L'objet Request

Accéder aux données de la requête

L'objet `Request` contient des informations sur la requête du client. Ces informations sont accessibles via plusieurs propriétés ou méthodes publiques :

- `request`: équivalent de `$_POST`;
- `query`: équivalent de `$_GET`;
- `cookies`: équivalent de `$_COOKIE`;
- `attributes`: pas d'équivalent - utilisé par l'application pour stocker d'autres données
- `files`: équivalent de `$_FILES`;
- `server`: équivalent de `$_SERVER`;
- `headers`: récupère une partie des valeurs de `$_SERVER`

(!) Chaque propriété est une instance de `ParameterBag`

L'objet Request

Les sessions

Le composant `HttpFoundation` possède un système de session très puissant et flexible conçu pour fournir une gestion de session que vous pouvez utiliser pour stocker des informations sur l'utilisateur entre les requêtes.

Les sessions Symfony sont conçues pour remplacer l'utilisation de la super globale `$_SESSION` et des fonctions associées (`session_start()`, `session_regenerate_id()`, `session_id()`, `session_name()` et `session_destroy()`).

Les Sessions

Accès et utilisation

La session est disponible via l'objet `Request` et le service `RequestStack`

```
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;

public function index(Request $request): Response
{
    $session = $request->getSession();

    // sauvegarde une valeur dans la session
    $session->set('foo-name', 'foo-value');

    // récupère une valeur par le nom
    $foo = $session->get('foo-name');

    // le deuxième argument est la valeur par défaut si la clé n'existe pas
    $bar = $session->get('bar-name', 'default-bar-value');
}
```

Les Sessions

Message Flash

Vous pouvez stocker des messages spéciaux, appelés messages « flash », sur la session de l'utilisateur.

Les messages flash sont destinés à être utilisés une seule fois : ils disparaissent automatiquement de la session dès que vous les affichez.

Cette fonctionnalité est particulièrement utile pour stocker les notifications des utilisateurs.

Les Sessions

Message Flash

Pour enregistrer un message « flash », on peut utiliser la méthode

`addFlash()` disponible depuis la classe `AbstractController`

```
$this->addFlash(type, mon message);
```

Pour afficher les messages « flash » dans twig:

```
{% for message in app.flashes(type) %}  
    <p style="color: limegreen">{{ message }}</p>  
{% endfor %}
```


Les Routes

Lorsque votre application reçoit une requête, elle appelle une action de contrôleur pour générer la réponse. La configuration de routage définit quelle action exécuter pour chaque URL entrante.

Il fournit également d'autres fonctionnalités utiles, comme la génération URL optimisées pour le référencement

Les Routes

Lorsque votre application reçoit une requête, elle appelle une action de contrôleur pour générer la réponse. La configuration de routage définit quelle action exécuter pour chaque URL entrante.

Il fournit également d'autres fonctionnalités utiles, comme la génération URL optimisées pour le référencement

Les routes peuvent être créées en YAML, XML, PHP ou en utilisant des attributs. Tous les formats apportent les mêmes fonctionnalités et performances.

Symfony recommande d'utiliser les attributs car il est pratique d'indiquer la route et le controller au même endroit.

Les Routes

Les paramètres de route

On peut définir une URL pour qu'elle ne change jamais (par exemple /formation). Cependant, il est courant de définir des routes où certaines parties sont variables. Par exemple, l'URL permettant d'afficher un article de blog inclura probablement le titre ou le slug.

Ex: /formation/symfony-initiation ou /formation/symfony-approfondissement

Dans les routes Symfony, les parties variables sont enveloppées dans { }. Par exemple, la route pour afficher le contenu de notre formation sera /formation/{slug}

Le nom de la partie variable (ici {slug}) sera transformé en une variable PHP et transmis en paramètre à notre controller.

Les routes peuvent définir n'importe quel nombre de paramètres.

Les Routes

Les paramètres de route

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class TrainingController extends AbstractController
{
    [...]

    #[Route('/formation/{slug}', name: 'training_show')]
    public function show($slug): Response
    {
        [...]

        return $this->render('training/show.html.twig');
    }
}
```

Les Routes

Valider des paramètres

Une application peut définir des routes qui peuvent être semblable:

- /formation/{slug}
- /formation/{page}

Par défaut les paramètres de route acceptent n'importe quelle valeur, il n'y a donc aucun moyen de différencier les deux routes.

Si on accède à l'URL /formation/symfony, les deux routes vont correspondre et Symfony utilisera la route qui a été définie en premier.

Pour pouvoir distinguer les deux routes ou pour valider un paramètre, on utilisera l'option `requirements`

```
#[Route('/formation/{page}', name: 'training_list', requirements: ['page' => '\d+'])]
```

L'option `requirements` définit les expressions régulières PHP.

Les Routes

Les valeurs par défaut

Quand on utilise les routes avec les annotations ou les attributs, les valeurs par défaut peut être définies dans les arguments de la méthode du contrôleur.

```
#[Route('/formation/{page}', name: 'training_list')]
public function list(int $page = 1): Response
{
    [...]
    return $this->render('training/list.html.twig');
}
```

Pour les autres formats de configuration, ils sont définis avec l'option defaults :

```
#[Route('/formation/{page}', name: 'training_list', defaults: ['page' => 1])]
public function list(int $page): Response
{
    [...]
    return $this->render('training/list.html.twig');
}
```

Les Routes

Priorité des routes

Symfony évalue les routes dans l'ordre dans lequel elles sont définies. Si la route correspond à plusieurs modèles différents, il se peut que cela empêcher d'autres routes de correspondre.

En YAML et XML, vous pouvez déplacer les définitions d'itinéraire vers le haut ou vers le bas dans le fichier de configuration pour contrôler leur priorité. Dans les routes définies comme annotations ou attributs PHP, cela est beaucoup plus difficile à réaliser.

On peut utiliser le paramètre `priority` pour contrôler leur priorité `#`

```
[Route('/route/{param}', name: 'route_name', priority: int_value)]
```

(!) Les routes avec une priorité plus élevée sont triées avant les routes avec une priorité plus faible. La valeur par défaut est 0.

Les Routes

Les méthodes HTTP

Par défaut, les routes peuvent être utilisées avec n'importe quel verbe HTTP (GET, POST, PUT, etc.). On utilise l'option `methods` pour restreindre une route à un ou plusieurs verbes

```
#[Route('/route/{param}', name: 'route_name', methods: ['GET', 'HEAD'])]
```


Les Routes

Special Parameters

En plus de vos propres paramètres, les routes peuvent inclure l'un des paramètres spéciaux suivants créés par Symfony :

`_controller`

Détermine le contrôleur et la méthode exécutés.

`_format`

Permet de définir le « format de demande » de l'objet Request. Ceci est utilisé pour des choses telles que le réglage du `Content-Type` de la réponse.

`_fragment`

Utilisé pour définir l'identifiant du fragment, qui est la dernière partie facultative d'une URL qui commence par un caractère # et est utilisée pour identifier une partie d'un document.

`_locale`

Utilisé pour définir la locale.

Les routes

Groupes de routes et préfixes

Il est courant qu'un groupe de routes partage des options communes. Pour ces cas, Symfony inclut une fonctionnalité de partage la configuration des routes.

Pour les définitions de route avec les attributs, on place la configuration commune dans l'attribut `#[Route]` du contrôleur.

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/formation', name: 'training_')]
class TrainingController extends AbstractController
{
    [...]
}
```

Débuguer les routes

`debug:router` liste toutes les routes de l'application.

```
php bin/console debug:router
```

Name	Method	Scheme	Host	Path
homepage	ANY	ANY	ANY	/
contact	GET	ANY	ANY	/contact

Voir la route en détail :

```
php bin/console debug:router app_lucky_number
```

Property	Value
Route Name	app_lucky_number
Path	/lucky/number/{max}
Options	compiler_class: Symfony\Component\Routing\RouteCompiler
	utf8: true

L'autre commande : `router:match` et cela montre quelle route correspond à un URL donné.

```
php bin/console router:match /lucky/number/8
```

```
[OK] Route "app_lucky_number" matches
```

Templating

Principe d'un moteur de template et présentation de Twig

Twig est un moteur flexible, rapide et sécurisé pour PHP

Le langage de création de template Twig permet d'écrire des modèles concis et lisibles, plus conviviaux pour les concepteurs Web et plus puissants que les modèles PHP.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome to Symfony!</title>
  </head>
  <body>
    <h1>{{ page_title }}</h1>

    {% if user.isLoggedIn %}
      Hello {{ user.name }}!
    {% endif %}

    {# ... #}
  </body>
</html>
```

Fichier de configuration Twig

Le fichier de configuration de twig se trouve dans `config/twig.yaml`.

répertoire de template

```
twig:
  default_path: '%kernel.project_dir%/templates'
```

variables globales

```
twig:
  globals:
    foo: 'bar'
    pi: 3.14
```

template de formulaire

```
twig:
  form_themes:
    - form_div_layout.html.twig
```

Les syntaxes de base

La syntaxe de twig est basé sur 3 structures:

- `{{ ... }}`, utilisé pour afficher le contenu d'une variable ou le résultat de l'évaluation d'une expression;
- `{% ... %}`, utilisé pour executer de la logique, comme une condition ou une boucle;
- `{# ... #}`, utilisé pour mettre des commentaires

Les syntaxes de base

La syntaxe de twig est basé sur 3 structures:

- `{{ ... }}`, utilisé pour afficher le contenu d'une variable ou le résultat de l'évaluation d'une expression;
 - `{% ... %}`, utilisé pour executer de la logique, comme une condition ou une boucle;
 - `{# ... #}`, utilisé pour mettre des commentaires
-

On trouve également sur une riche bibliothèque de **tags**, **filtres**, **fonctions**, et **tests**, complété par Symfony pour faciliter l'utilisation des composants du framework (formulaire, traduction, sécurité, etc)

(!) Il est possible de créer ces propres tags, filtres, fonctions et tests

Les syntaxes de base

Rendu d'un template depuis un contrôleur

Si votre contrôleur étends la classe `AbstractController`, on peut utiliser une des méthodes helper

- `render()`: Retourne un Objet `Response` avec le contenu créé par le template
- `renderView()`: Retourne uniquement le contenu créé par le template

Le second paramètre des méthodes `render()` et `renderView()` permet de transmettre des paramètres au template.

Les syntaxes de base

Afficher des valeurs

```
1 + 2 = {{ 1 + 2 }}
```

```
Bonjour {{ name }}
```

Les syntaxes de base

Structure iterative

```
{% for value in 1..4 %}  
  - {{ value }}  
{% endfor %}
```

for ... else

```
{% for value in [] %}  
  - {{ value }}  
{% else %}  
  Aucune valeur  
{% endfor %}
```

key and value

```
{% for key, value in {'firstname': 'John', 'lastname': 'Doe'} %}  
  - {{ key }} : {{ value }}  
{% endfor %}
```

Les syntaxes de base

Structure conditionnelle

```
{% if name == "John" %}  
    Bonjour John  
{% endif %}
```

if ... elseif ... else ...

```
{% set temperature = 31 %}  
  
{% if temperature > 30 %}  
    Il fait très chaud  
{% elseif temperature > 20 %}  
    Il fait chaud  
{% else %}  
    Il fait pas chaud  
{% endif %}
```

Les syntaxes de base

Les tests et opérateur logique

En plus des tests classiques de PHP (`==`, `===`, `<`, `>`, `<=`, `>=`, `!=`, `!==`), Twig met à disposition un ensemble d'autres comparateurs

- `is`
- `in`
- `starts with` / `ends with`
- `matches`
- `etc...`

Ainsi qu'un ensemble de test

- `defined`
- `divisibleby`
- `empty`
- `even` / `odd`
- `null`
- `same as`

Les syntaxes de base

Les tests et opérateur logique

```
{% if [] is empty %}  
  ...  
{% endif %}  
  
{% if 5 in [1, 2, 3, 4, 5] %}  
  ...  
{% endif %}  
  
{% if 10 is divisible by(3) %}  
  ...  
{% endif %}
```

Les syntaxes de base

Auto échappement

Il y a deux approches pour l'échappement: l'échappement manuel ou automatique. L'échappement automatique est activé par défaut.

```
{{ "<b>Hello World</b>" }} {# affiche: <b>Hello World</b> #}
```

Désactiver l'échappement

Pour désactiver l'échappement automatique d'une chaîne, on peut utiliser le filtre `raw`

```
{{ "<b>Hello World</b>|raw" }} {# affiche: Hello World #}
```

Les syntaxes de base

Les filtres

Les variables peuvent être modifiées par des filtres. Les filtres sont séparés de la variable par un symbole pipe (|).

```
{{ 'hello'|upper }}
```

Plusieurs filtres peuvent être chaînés. La sortie d'un filtre est appliquée au suivant.

```
{{ '<b>hello</b>'|striptags|title }}
```

Pour appliquer un filtre sur plusieurs ligne, on peut l'encadrer avec le tag `apply`

```
{% apply upper %}  
  Lorem ipsum dolor sit amet, consectetur adipiscing elit.  
{% endapply %}
```


Les syntaxes de base

Les fonctions

Twig met à disposition un ensemble de fonction. Les fonctions sont appelées par leur nom suivi de parenthèses (()) et peuvent avoir des arguments.

```
{% dump("hello world") %} ou {{ dump("hello world") }}
```

```
{{ max(1, 3, 2) }}
```

Twig permet également de créer des fonctions à l'aide du tag `macro`. Les macros doivent être importées avant d'être utilisé.

```
{% import _self as macro %}  
{% import "file_macro.twig" as macro %}
```



```
{% macro hello(name = "World") %}  
    Hello {{ name }}  
{% endmacro %}
```

Les syntaxes de base

Arguments nommés

Les arguments nommés peuvent être utilisé dans les **fonctions**, les **filtres** et les **tests**.

```
{% for i in range(low=1, high=10, step=2) %}  
  {{ i }},  
{% endfor %}
```

Les arguments nommés vous permettent entre autre d'ignorer certains arguments pour lesquels vous ne souhaitez pas modifier la valeur par défaut

```
{# sans argument nommé #}  
{{ "now"|date(null, "Europe/Paris") }}
```



```
{# avec argument nommé #}  
{{ "now"|date(timezone="Europe/Paris") }}
```

Les syntaxes de base

Accéder aux valeurs d'un tableau ou d'un objet

On utilise le point (.) pour accéder aux attributs d'une variable (méthodes ou propriétés d'un objet PHP, ou éléments d'un tableau PHP)

```
{% set contact = { 'name': "John Doe", 'email': "john.doe@gmail.com" }  
{{ contact.name }} - {{ contact['email'] }}
```

(!) Twig supporte la syntaxe PHP (`contact['email']`) pour accéder aux données d'un tableau

Lors d'un appel à `foo.bar`, les opérations suivantes sont effectuées en PHP:

- vérifie si `foo` est un tableau et `bar` un élément du tableau
- sinon, si `foo` est un objet, vérifie si
 - `.code[bar]` est une propriété valide
 - `.code[bar]` est une méthode valide
 - `.code[getBar]` est une méthode valide
 - `.code[isBar]` est une méthode valide
 - `.code[hasBar]` est une méthode valide
- sinon, retourne `null`

Référencer les pages

créer des liens hypertexte

Pour créer des liens hypertexte dans twig, on peut utiliser

- `path()` pour créer des urls relative
- `url()` pour créer des urls absolue

```
<a href="{{ path('training_index') }}"> ... </a>
```

```
<a href="{{ url('training_index') }}"> ... </a>
```

Les fonctions `path()` et `url()`, acceptent un 2ème paramètre pour indiquer les paramètres de route ou de requête

```
<a href="{{ path('training_list', { 'page': 1 }) }}"> ... </a>
```

Référencer des ressources JS, CSS et images

Pour créer des liens vers des fichiers CSS, JS ou images, Symfony (twig-bridge) met à disposition la fonction `asset()` pour aider à générer les URL.

```


<link href="{{ asset('css/style.css') }}" rel="stylesheet"/>

<script src="{{ asset('js/script.js') }}"></script>
```

La fonction `asset()` crée des **URLs relative** en allant chercher les ressources demandées directement dans le répertoire `public/`

Pour créer des URLs absolues, on peut utiliser la fonction `absolute_url()`

```


<link rel="shortcut icon" href="{{ absolute_url('favicon.png') }}">
```

Factoriser les templates

Héritage

La partie la plus puissante de Twig est l'héritage des templates. L'héritage de template permet de créer un template de base « squelette » qui contient tous les éléments communs de votre site.

Dans le template de base, on utilise les tags `block` pour créer des zones dynamique que les enfants pourront alimenter.

Par défaut le template de base s'appelle `base.html.twig`

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>{% block title %}Welcome!{% endblock %}</title>

    {% block stylesheets %}{% endblock %}

    {% block javascripts %}{% endblock %}
  </head>
  <body>
    <header>{% block body %}Symfony{% endblock %}</header>
    <main>{% block body %}{% endblock %}</main>
  </body>
</html>
```

Factoriser les templates

Héritage

Pour mettre en place un héritage de template, on utilise le tag `extends`. Il indique à Twig que le template « étend » un autre template.

Le tag `extends` doit être le premier tag du template.

```
{% extends "base.html" %}

{% block title %}Mon title{% endblock %}

{% block body %}[...]{% endblock %}
```

Si un template enfant ne définit pas un `block`, la valeur du parent est utilisée par défaut.

Il est possible d'afficher dans un `block` enfant, le contenu d'un `block` parent en utilisant la fonction `parent()`.

```
{% extends "base.html" %}

{% block header %}
  {{ parent() }}
  <h2>Version 6</h2>
{% endblock %}
```

Factoriser les templates

Inclusion

La fonction `include` est utile pour inclure un template dans un autre.

```
{{ include('partial/_template.html.twig') }}
```

(!) Le préfixe `_` est facultatif. C'est une convention utilisée pour mieux différencier les templates complets des fragments de templates

Par défaut, les templates inclus ont accès au même contexte que le template qui les inclut. Cela signifie que toute variable définie dans le template principal sera également disponible dans le template inclus

Factoriser les templates

Sous-requête

Les sous-requêtes se font en incluant le résultat de l'exécution d'un controller dans une vue (fichier twig), à l'aide des fonctions `render()` et `controller()`

(!) Par convention les fragments de template sont préfixés par un `_` pour les différencier des templates complets

Si le contrôleur est associé à une route, on peut utiliser les fonctions `path()` ou `url()`

```
{{ render( path('route_name') ) }}  
{{ render( url('route_name') ) }}
```

Si on ne souhaite pas exposer le contrôleur avec une URL publique, on peut utiliser la fonction `controller()` pour définir le contrôleur à exécuter

```
{{ render( controller('App\\Controller\\TrainingController::nextSession') ) }}
```

Base de données (Doctrine)

Comprendre le rôle d'un ORM

ORM : Object-Relational Mapping

Le principe est de pouvoir simplifier l'accès aux BDD en proposant d'utiliser des objets pour manipuler les données relationnelles plutôt que d'y accéder directement.

- Réduction du code (création, maintenance)
- Homogénéité du code objet
- On n'écrit pas de SQL
- Changer de base de données sans reprendre le code
- Diminuer le temps de développement

Tous les outils nécessaires pour utiliser les bases de données sont présents dans Symfony grâce à Doctrine. Doctrine est une librairie PHP qui permet d'utiliser des bases de données comme MySQL et PostgreSQL et aussi du NoSQL comme MongoDB.

Installation

Pour vérifier si doctrine est déjà installé, dans la console on lance la commande `php bin/console doctrine`

Sinon on installe Doctrine depuis le package ORM de Symfony

```
composer require symfony/orm-pack
```

Configurer la BDD

La connexion à la BDD est enregistrée dans une variable d'environnement DATABASE_URL.

```
# .env (or override DATABASE_URL in .env.local to avoid committing your changes)
# customize this line!
DATABASE_URL="mysql://db_user:db_password@127.0.0.1:3306/db_name?serverVersion=5.7"
# to use mariadb:
DATABASE_URL="mysql://db_user:db_password@127.0.0.1:3306/db_name?serverVersion=mariadb-10.5.8"
# to use sqlite:
# DATABASE_URL="sqlite:///kernel.project_dir%/var/app.db"
# to use postgresql:
# DATABASE_URL="postgresql://db_user:db_password@127.0.0.1:5432/db_name?serverVersion=11&charset=utf8"
# to use oracle:
# DATABASE_URL="oci8://db_user:db_password@127.0.0.1:1521/db_name"
```

Créer la BDD

A partir de maintenant, la création, modification, et autres manipulations des BDD se feront soit par des commandes terminal, soit par de la manipulation d'objets dans Symfony, on n'écrira pas de SQL

Maintenant que les paramètres de connexion sont configurés, Doctrine peut créer le db_name de la BDD :

```
php bin/console doctrine:database:create
```

Il existe plus d'options dans le fichier config/packages/doctrine.yaml, dont la version de serveur (5.7 si on utilise MySQL 5.7), ce qui peut affecter la façon dont Doctrine fonctionne.

(!) Il existe d'autres commandes Doctrine, en utilisant `php bin/console list doctrine` pour voir la liste des commandes.

Créer une classe d'entité

Pour **créer ou mettre à jour** une entité, on utilise une commande du composant

Maker

```
php bin/console make:entity
```

La commande posera une série de questions pour créer notre entité:

- nom de l'entité
- nom des champs
- type de champs
- longueur (**varchar**)
- champs optionnel (**nullable**)

La commande va créer 2 types de fichier:

- L'entité : enregistrée dans le répertoire `src/Entity/`
- Le repository: enregistré dans le répertoire `src/Repository/`

Utiliser les migrations

Créer/modifier le schéma

Pour créer le schéma de base de données, on peut utiliser les commandes doctrine (amélioré par Symfony):

- `php bin/console doctrine:schema:create` : créer le schéma de base de données
- `php bin/console doctrine:schema:update` : mettre à jour le schéma de base de données

ou passer par les migrations du bundle `DoctrineMigrationsBundle`:

- `php bin/console make:migration` : créer un fichier de migration contenant les requêtes SQL nécessaire pour mettre à jour le base de données
- `php bin/console doctrine:migrations:migrate` : execute tous les fichiers de migration qui n'ont pas encore été joué dans la base de données

Modifier une classe d'entité

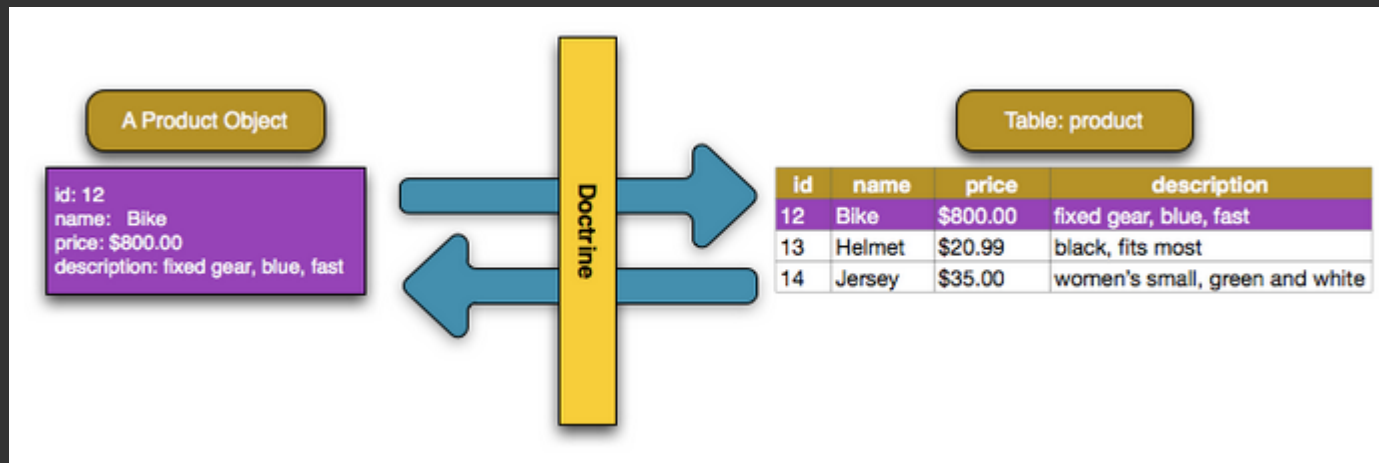
Pour modifier une entité existante, on utilise la même commande que pour la création.

```
php bin/console make:entity
```

Si l'entité existe déjà, la commande proposera d'ajouter de nouveaux champs. Le comportement sera similaire à celui de la création.

Une fois l'entité mis à jour, on peut à nouveau créer un fichier de migration et l'exécuter sur la base de données

Charger et enregistrer un objet en base



Doctrine met à disposition 2 types de classes charger et enregistrer un objet dans la base

- l'entity manager: se charge de l'enregistrement (Objet ==> Bdd)
- le repository: se charge de l'extraction (Objet <== Bdd)

Enregistrer un objet en base

Pour enregistrer un objet en base, nous devons d'abord récupérer la classe (service) `EntityManager` (`EntityManagerInterface`).

- `persist`: Inscrit une entité dans la mémoire de doctrine
- `flush`: Exécute les requêtes

Si la méthode `flush` ne fonctionne pas, une exception est lancée par `Doctrine\ORM\ORMException`

```
public function createEntity(EntityManagerInterface $entityManager): Response
{
    $entity = new Entity();
    $entity->setProperty(...);
    $entity->setProperty(...);

    $entityManager->persist($entity);
    $entityManager->flush();

    // ...
}
```

(!) Pour la mise à jour d'une entité, le travail est similaire. Doctrine sait s'il doit mettre à jour ou insérer l'entité

Charger des objets depuis la base

Pour charger un objet depuis la base, nous devons d'abord récupérer la classe (service) `Repository` de notre entité.

(!) Toutes les entités ont un repository qui leurs sont associé

Les classes `Repository` permettent de créer des méthodes d'accès aux données personnalisées (selectionner les champs, conditions, tries, jointures, etc).

Les `Repository` exposent 4 méthodes par défaut:

- `find`: rechercher une valeur avec sa clef primaire
- `findAll`: rechercher toutes les valeurs
- `findOneBy`: rechercher une valeur en fonction de la valeur d'un ou plusieurs colonnes
- `findBy`: rechercher toutes les valeurs en fonction de la valeur d'un ou plusieurs colonnes

```
public function createEntity(TrainingRepository $trainingRepository): Response
{
    $trainings = $trainingRepository->findAll();
    $trainings = $trainingRepository->findBy(['field' => 'value']);
    // ...
}
```

Charger des objets depuis la base

Pour construire des requêtes d'accès aux données plus spécifique, on peut créer de nouvelles méthodes dans la classe `Repository`.

Dans le repository les requêtes peuvent être écrites de 2 façons:

- En DQL (Doctrine Query Language): Permet d'écrire des requêtes en utilisant le SQL, mais en faisant référence à des objets PHP.
- Avec l'objet `QueryBuilder`: Une manière orientée objet pour écrire des requêtes.

Charger des objets depuis la base

DQL

Pour créer une requête en DQL, il faut utiliser la méthode `createQuery()`.

```
// \App\Repository\TrainingRepository.php
public function findAllByDuration(int $duration): array
{
    $entityManager = $this->getEntityManager();

    $query = $entityManager->createQuery(
        'SELECT t
        FROM App\Entity\Training t
        WHERE t.duration > :duration'
    )->setParameter('duration', $duration);

    // returns an array of Training objects
    return $query->getResult();
}
```

Charger des objets depuis la base

QueryBuilder

Il est recommandé d'utiliser le `QueryBuilder` lorsque les requêtes sont construites dynamiquement.

Méthode de la classe `QueryBuilder` pour construire les requêtes:

- `select / addSelect`
- `from`
- `where / andWhere / orWhere`
- `join / leftJoin / innerJoin`
- `order / addOrder`
- `groupBy / addGroupBy`
- `having / andHaving / orHaving`

Charger des objets depuis la base

QueryBuilder: Récupérer les résultats

Pour obtenir la requête, on utilise la méthode `getQuery()`

Méthode récupérer des résultats:

- `getResult`
- `getOneOrNullResult`
- `getSingleResult`
- `getArrayResult`
- `getScalarResult`
- `getSingleScalarResult`

Charger des objets depuis la base

QueryBuilder

```
// \App\Repository\TrainingRepository.php
public function findAllByDuration(int $duration): array
{
    return $this->createQueryBuilder('t')
        ->where('t.duration > :duration')
        ->setParameter('duration', $duration)
        ->getQuery()
        ->getResult();
}
```

Charger des objets depuis la base

Liaison des paramètres à la requête

Doctrine prend en charge la liaison dynamique des paramètres à votre requête, de la même manière que pour les requêtes préparées.

Les valeurs de remplacement peuvent être utilisées avec des chaînes de caractère ou des nombres.

```
// \App\Repository\TrainingRepository.php
public function findAllByDuration(int $duration): array
{
    return $this->createQueryBuilder('t')
        ->where('t.duration > ?1')
        ->setParameter(1, $duration)
        ->getQuery()
        ->getResult();
}
```

```
// \App\Repository\TrainingRepository.php
public function findAllByDuration(int $duration): array
{
    return $this->createQueryBuilder('t')
        ->where('t.duration > :duration')
        ->setParameter('duration', $duration)
        ->getQuery()
        ->getResult();
}
```

Charger automatiquement depuis la route

EntityValueResolver

(!) EntityValueResolver a été ajouté dans DoctrineBundle 2.7.1

EntityValueResolver permet déléguer à Symfony l'exécution des requêtes.

```
// ...
class ProductController extends AbstractController
{
    #[Route('/product/{id}')]
    public function show(Product $product): Response
    {
        // ...
    }
}
```

Symfony utilise le paramètre {id} dans la route pour faire la requête (`find()`) et fournir en paramètre du contrôleur une instance de l'entité demandé.

Si l'entité n'est pas trouvé, une page 404 sera générée.

Charger automatiquement depuis la route

EntityValueResolver

Il est également possible d'utiliser le composant `ExpressionLanguage` pour écrire des expressions.

```
#[Route('/product/{product_id}')]
public function show(
    #[MapEntity(expr: 'repository.find(product_id)')]
    Product $product
): Response {
}
```

Dans l'expression, la variable `repository` fera référence au repository de notre entité.

Tous les paramètres dynamiques (ex: `product_id`) de la route seront également disponibles.

Mis en place de relations/associations

Il existe 3 types de relations :

- OneToOne
- ManyToOne / OneToMany
- ManyToMany

La relation entre 2 entités peut se faire en utilisant la commande `make:entity`, en précisant le type de relation souhaité quand la commande demande le type du champs à ajouter

```
Field type (enter ? to see all types) [string]:
> ?
[...]
Relationships/Associations
* relation or a wizard 🧙 will help you build the relation
* ManyToOne
* OneToMany
* ManyToMany
* OneToOne
```

(!) Le type **relation** est un helper qui nous guide dans la mise en place de la relation

Mis en place de relations/associations

Une relation peut s'utiliser soit de façon:

- **unidirectionnelle**: seule l'entité qui contient la relation peut avoir accès aux informations de l'entité jointe
- **bidirectionnelle**: les 2 entités peuvent avoir accès aux informations de l'autre

Dans le cas d'une relation bidirectionnelle, il faudra préciser la manière dont les champs sont liés:

```
// App\Entity\Session.php
#[ORM\ManyToOne(inversedBy: 'sessions')]
private Training $training

// App\Entity\Training.php
#[ORM\OneToMany(mappedBy: 'training', targetEntity: Training::class)]
private Collection $sessions
```

Mis en place de relations/associations

OneToOne / ManyToOne

- Ajoute une clé étrangère dans la table de l'entité dans laquelle elle est définie.

OneToMany

- Est obligatoirement une relation bidirectionnelle.
- Utilise une instance respectant l'interface `Doctrine\Common\Collections\Collection`
- Implémente un constructeur pour initialiser la **Collection**
(`Doctrine\Common\Collections\ArrayCollection`)

ManyToMany

- Doctrine prends en charge la création de la table intermédiaire.

Les formulaires

Principes Symfony des formulaires

Installation

```
composer require symfony/form
```

Utilisation

1. Créer le formulaire dans un controller ou une classe dédiée
2. Créer un template dans lequel le formulaire sera vu et utilisé
3. Traiter les données récupérée par le formulaire

Construire le formulaire et l'association à une classe de données

Pour créer un formulaire : `php bin/console make:form`

La commande pose 2 questions:

- Le nom du formulaire
- Veut on rattacher une entité à ce formulaire

Le formulaire est créé dans le répertoire `src/Form/`

(!) Les classes de formulaire sont suffixé par la valeur **Type** (ex: `ContactType`)

Les classes de formulaire

Un formulaire est composé de champs, chacun étant construit à l'aide d'un type de champ (par exemple `TextType`, `ChoiceType`, etc.).

Par défaut, Symfony fournit une large liste de types de champs

[Liste des types de champs](#)

Les champs de formulaire peuvent être configurés avec un ensemble d'options spécifique à chaque type de champs

- `label`: définit le label du champs
- `attr`: permet d'ajouter les attributs HTML du champ de formulaire
- `help`: permet de définir un texte d'aide
- `required`: permet de définir si on utilise l'attribut HTML5 **required**
- `mapped`: permet de ne pas mapper le champs lors de la soumission
- `disabled`: permet de désactiver le champs en lecture et écrire
- etc

Rendre le formulaire en HTML

Dans le contrôleur

Avant d'afficher notre formulaire dans twig, il faut commencer par le créer dans notre formulaire.

```
$form = $this->createForm(FQCN*, defaultValue);
```

* FQCN: Fully Qualified Class Name (ex: Training::class). Nom complet de la classe avec le namespace: \App\Entity\Training

Rendre le formulaire en HTML

Dans la vue

Pour afficher un formulaire dans twig, Symfony (`TwigBridge`) met à disposition un ensemble de fonction qui vont se charger de faire le rendu de nos formulaires

- `form()`
- `form_start()`
- `form_end()`
- `form_row()`
- `form_label()`
- `form_widget()`
- `form_errors()`
- `form_help()`
- `form_rest()`

Réceptionner et valider les données

Lors du chargement initial de la page dans un navigateur, le formulaire n'a pas encore été soumis et la méthode `isSubmitted()` renvoie `false`.

```
if ($form->isSubmitted && $form->isValid()) {  
    [...]  
}
```

Pour réceptionner des données soumissionnées par un formulaire, on utilise la méthode `$form->handleRequest()` qui détecte la soumission et affecte les valeurs à l'entité rattachée au formulaire.

```
$training = new Training();  
$form = $this->createForm(TrainingType::class, $training);  
$form->handleRequest($request);
```

Une fois que l'entité aura été affectée, elle sera ensuite validée. Si l'entité n'est pas valide, `isValid()` renvoie `false` et le formulaire s'affiche à nouveau, mais cette fois avec des erreurs de validation (HTTP 422 Unprocessable Content).

(!) Rediriger un utilisateur après une soumission réussie du formulaire est une bonne pratique qui empêche l'utilisateur de pouvoir appuyer sur le bouton « Actualiser » de son navigateur et de republier les données.

Réceptionner et valider les données

Afin de pouvoir valider une entité, il faut lui appliquer un ensemble de règles appelé **contraintes**.

Les contraintes peuvent être appliqué directement sur une entité ou en utilisant l'option **constraints** sur un formulaire.

```
namespace App\Entity;

use Symfony\Component\Validator\Constraints as Assert;

class Training
{
    #[Assert\NotBlank]
    public string $title;

    ...
}
```

[Liste des contraintes présentent par défaut dans Symfony.](#)

Créer ses propres validateurs

Il est possible de créer ces propres classes de contraintes et validations.

Pour créer une contraintes on peut utiliser la commande:

```
php bin/console make:validator
```

La commande permet de créer 2 fichiers: la classe de contrainte et la classe de validations

Créer ses propres validateurs

Une contrainte est une classe PHP qui hérite de la classe `Constraint` de Symfony. Si on souhaite utiliser cette classe comme un attribut PHP, on peut lui ajouter l'attribut `#[\Attribute]`

```
namespace App\Validator;

use Symfony\Component\Validator\Constraint;

#[\Attribute]
class MyConstraint extends Constraint
{
    public string $message = 'My custom error message !!!';

    public function __construct(string $message = null, array $groups = null, $payload = null)
    {
        parent::__construct([], $groups, $payload);

        $this->message = $message ?? $this->message;
    }
}
```

Créer ses propres validateurs

Un validateur est une classe PHP qui hérite de la classe `ConstraintValidator` de Symfony. Cette classe implémente la méthode `validate` dans laquelle on définit les règles de validations et ou en cas d'erreur on ajoute les violations.

```
namespace App\Validator;

use Symfony\Component\Validator\Constraint;
use Symfony\Component\Validator\ConstraintValidator;

class MyConstraintValidator extends ConstraintValidator
{
    public function validate($value, Constraint $constraint)
    {
        if (null === $value || '' === $value) {
            return;
        }

        // TODO: implement the validation here
        $this->context->buildViolation($constraint->message)
            ->setParameter('{{ value }}', $value)
            ->addViolation();
    }
}
```

Personnaliser le rendu (theming)

Symfony est livré avec plusieurs thèmes de formulaires intégrés utilisable avec certains des frameworks CSS.

Les thèmes se configurent dans le fichier `'config/packages/twig.yaml'` avec l'option `form_themes`

- `form_div_layout.html.twig` - thème par défaut
- `form_table_layout.html.twig`
- `bootstrap_3_layout.html.twig`
- `bootstrap_3_horizontal_layout.html.twig`
- `bootstrap_4_layout.html.twig`
- `bootstrap_4_horizontal_layout.html.twig`
- `bootstrap_5_layout.html.twig`
- `bootstrap_5_horizontal_layout.html.twig`
- `foundation_5_layout.html.twig`
- `foundation_6_layout.html.twig`
- `tailwind_2_layout.html.twig`

La sécurité

Principe de fonctionnement: authentification et autorisation

Des outils de sécurité existent dans Symfony : HTTP, cookies de session sécurisés, authentification json et protection CSRF. Le `SecurityBundle` propose les fonctionnalités d'**authentification** et d'**autorisation** pour sécuriser l'application.

Installer le SecurityBundle:

```
composer require symfony/security-bundle
```

Les permissions sont liées à des utilisateurs, il faut donc créer une classe d'utilisateur.

Créer sa classe d'utilisateur, le fournisseur et gérer l'encodage des mot-de-passes

Créer la classe d'utilisateur

Par défaut symfony propose 4 services pour gérer des utilisateurs:

1. depuis la base de données (Entity User Provider)
2. depuis un serveur LDAP (LDAP User Provider)
3. depuis le fichier security.yaml (Memory User Provider)
4. en combinant plusieurs services (Chain User Provider)

Memory User Provider

```
# config/packages/security.yaml
security:
  providers:
    in_memory_provider:
      memory:
        users:
          admin: { password: '...', roles: ['ROLE_ADMIN'] }
          user: { password: '...', roles: ['ROLE_USER'] }

# ...
```

Créer la classe d'utilisateur

Entity User Provider

Pour créer des utilisateurs stockés dans notre base de données, on peut utiliser la commande `make:user` qui guide pas à pas dans la création de l'entité utilisateur.

```
# config/packages/security.yaml
security:
  providers:
    users:
      entity:
        class: App\Entity\User

# ...
```


Créer la classe d'utilisateur

Entity User Provider

```
php bin/console make:user
The name of the security user class (e.g. User) [User]:
> User

Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
> yes

Enter a property name that will be the unique "display" name for the user
(e.g. email, username, uuid) [email]:
> email

Will this app need to hash/check user passwords? Choose No if passwords are not needed or will
be checked/hashed by some other system (e.g. a single sign-on server).

Does this app need to hash/check user passwords? (yes/no) [yes]:
> yes

created: src/Entity/User.php
created: src/Repository/UserRepository.php
updated: src/Entity/User.php
updated: config/packages/security.yaml
```

Gérer le fournisseur (user provider)

Symfony ajoute et utilise aussi un user provider, dans le fichier `config/packages/security.php`. Le user provider sait recharger un utilisateur depuis une ressource (BDD par ex.) par rapport à un identifiant.

Il est utilisé pour

- Charger un utilisateur basé sur son identifiant (authentification)
 - Recharger un utilisateur depuis la session
- Au début de chaque requête, le utilisateur est chargé depuis la session. Le user provider "raffraîchit" l'utilisateur pour être sûr que toutes les informations sont à jour.

Encodage des mots de passe

Le `SecurityBundle` fournit des fonctions d'encodage et de vérification des mots de passe.

Le choix de l'algorithme pour encoder les mots de passe se configure dans le fichier `security.yaml`

```
# config/packages/security.yaml
security:
    # ...
    password_hashers:
        # Use native password hasher, which auto-selects and migrates the best
        # possible hashing algorithm (which currently is "bcrypt")
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
```

(!) Il est possible d'utiliser la commande `php bin/console security:hash-password` pour hasher un mot de passe

<https://symfony.com/doc/current/security/passwords.html#supported-algorithms>

Encodage des mots de passe

Dans le cas où on utilise une classe `User`, elle doit implémenter l'interface `PasswordAuthenticatedUserInterface`, pour permettre à Symfony de récupérer le mot de passe sans erreur.

Dans un controller, pour hasher le mot de passe utilisateur, on utilise le service `UserPasswordHasherInterface`

```
public function index(UserPasswordHasherInterface $passwordHasher): Response
{
    // ... e.g. get the user data from a registration form
    $user = new User(...);
    $plaintextPassword = ...;

    // hash the password (based on the security.yaml config for the $user class)
    $hashedPassword = $passwordHasher->hashPassword(
        $user,
        $plaintextPassword
    );
    $user->setPassword($hashedPassword);

    // ...
}
```

Authentifier l'utilisateur sur un contexte de pare-feu

La section firewalls de `config/packages/security.yaml` est très importante. C'est le système d'authentification, il définit quels sont les parties de l'application qui sont sécurisées et comment les utilisateurs pourront s'authentifier.

Un seul firewall est actif par requête. (en dev c'est un faux firewall)

Le firewall

- `main` vérifie toutes les urls
- Il peut avoir plusieurs modes d'authentification

Contrôler l'accès sur des rôles au niveau du pare-feu, des routes ou dans le code

Restreindre les accès à routes peut se faire de plusieurs façon dans l'application:

Dans le fichier `security.yaml`, dans la partie `access_control` en indiquant le pattern pour une route de l'application ainsi que le ou les roles nécessaire pour y accéder.

```
security:
  ...

  # Easy way to control access for large sections of your site
  # Note: Only the *first* access control that matches will be used
  access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/profile, roles: ROLE_USER }
```

Contrôler l'accès sur des rôles au niveau du pare-feu, des routes ou dans le code

Sur les contrôleurs soit en utilisant l'attribut `#[IsGranted]` sur la classe ou sur une méthode, ou en utilisant la méthode `denyAccessUnlessGranted` de la classe abstraite `AbstractController`.

L'attribut `#[IsGranted]` et la méthode `denyAccessUnlessGranted` lance une exception `AccessDeniedException` avec un code de réponse HTTP 403.

```
#[Route('/dashboard', name: 'app_admin_dashboard')]
#[IsGranted('ROLE_ADMIN')]
public function dashboard(): Response
{
    $this->denyAccessUnlessGranted('ROLE_ADMIN');

    [...]
}
```

(!) Il existe également une méthode `isGranted` dans la classe `AbstractController` qui retourne une valeur booléenne

Contrôler l'accès sur des rôles au niveau du pare-feu, des routes ou dans le code

Enfin il est possible également de restreindre l'affichage de certains éléments dans la vue à l'aide de la fonction twig `is_granted()`

```
{% if not app.user %}
    <a href="{{ path('authentication_login') }}" class="nav-link">Connexion</a>
{% else %}
    <span class="nav-link disabled">Bonjour {{ app.user.username }}</span>
    <a href="{{ path('authentication_logout') }}" class="nav-link">Déconnexion</a>

    {% if is_granted('ROLE_ADMIN') %}
        <a href="{{ path('admin_post_index') }}" class="nav-link">Admin Articles</a>
    {% endif %}
{% endif %}
```


Créer des règles personnalisées avec des voters

Les `voters` constituent le moyen le plus puissant de Symfony pour gérer les autorisations. Ils vous permettent de centraliser toute la logique d'autorisation, puis de les réutiliser à de nombreux endroits.

Cependant, si vous ne réutilisez pas les autorisations ou si vos règles sont basiques, vous pouvez toujours mettre cette logique directement dans votre contrôleur.

Les `voters` sont appelés à chaque fois que vous utilisez la méthode `isGranted()` ou `denyAccessUnlessGranted()`

Symfony prend en considérations la réponse de tous les `voters` et prend une décision (d'autoriser ou de refuser l'accès) selon la stratégie définie.

Créer des règles personnalisées avec des voters

Stratégies

Il existe 4 stratégies par défaut:

- **affirmative** (valeur par défaut): accorde l'accès dès qu'un `voter` accorde l'accès
- **consensuelle**: accorde l'accès s'il y a plus de `voters` accordant l'accès que refusant. En cas d'égalité, la décision est basée sur l'option de configuration `allow_if_equal_granted_denied` (par défaut sur true)
- **unanime**: accorde l'accès si tous les `voters` accorde l'accès
- **prioritaire**: accorde ou refuse l'accès au premier `voter` qui ne s'abstient pas, en fonction de sa priorité

Créer des règles personnalisées avec des voters

Pour créer un `voter` on utilise la commande `php bin/console make:voter`.

Les `voters` sont enregistrés dans le répertoire `src/Security/Voter/`

Les `voters` utilisent 2 méthodes:

- `supports`: Détermine si le `voter` peut s'exprimer en fonction des paramètres transmis
- `voteOnAttribute`: Détermine si le `voter` autorise (`return true`) ou refuse (`return false`) l'accès. C'est dans cette méthode que l'on définit la logique métier

Créer des règles personnalisées avec des voters

Tester les rôles

Pour tester le rôle d'un utilisateur dans un voter, on fait appel au service `Security`.

```
use Symfony\Bundle\SecurityBundle\Security;

class ExampleVoter extends Voter
{
    public function __construct(
        private Security $security,
    ) { }

    // ...

    protected function voteOnAttribute($attribute, mixed $subject, TokenInterface $token): bool
    {
        if ($this->security->isGranted('ROLE_SUPER_ADMIN')) {
            return true;
        }

        // ... all the normal voter logic
    }
}
```

Internationalisation

Configurer la langue, utiliser des catalogues

L'internationalisation (i18n), est le processus qui permet de traduire les chaînes de caractère du site dans la langue de l'utilisateur en s'appuyant sur la `locale` choisi par l'utilisateur.

(!) Le terme `locale` fait référence à la langue et au pays de l'utilisateur. Il est recommandé d'utiliser le code de langue ISO 639-1, un underscore (_), puis le code pays ISO 3166-1 alpha-2 (exemple: `fr_FR` pour le français de France)

Les traductions peuvent être organisées en groupes, appelés domaines. Par défaut, tous les messages utilisent le domaine de `messages`.

Configurer la langue, utiliser des catalogues

La configuration se fait dans le fichier `config/packages/translation.yaml`

```
framework:
    default_locale: fr
    translator:
        default_path: '%kernel.project_dir%/translations'
        fallbacks:
            - en
```

- `default_locale`: définit la `locale` par défaut utilisée par l'application
- `translator.fallbacks`: définit les `locales` à utiliser si une traduction n'est pas trouvée dans le fichier de `locale` par défaut

Configurer la langue, utiliser des catalogues

Pour changer de `locale` et permettre la traduction de notre interface, la politique la plus efficace consiste à inclure les paramètres régionaux dans l'URL à l'aide du [paramètre spécial d'url](#) `_locale`

```
#[Route(  
  path: '/{_locale}/formation',  
  name: 'training_index',  
  requirements: [  
    '_locale' => 'en|fr|de',  
  ],  
)]
```


Rédiger les catalogues de messages

La traduction du texte est effectuée par le service `Translator`. Pour traduire un bloc de texte (message), utilisez la méthode `trans()`.

Pour traduire un message, Symfony s'appuie généralement sur un fichier contenant la liste des traductions à utiliser par rapport à la `locale`.

`messages.fr.yaml` ou `messages.en.yaml`

Le fichier de traduction peut être utilisé sous plusieurs formats:

- `yaml`
- `xlif`
- `php`
- `csv`
- `json`
- `ini`
- `etc`

(!) Il est recommandé d'utiliser YAML pour des projets simples et d'utiliser XLIFF si vous générez des traductions avec des programmes ou des équipes spécialisées

Rédiger les catalogues de messages

Dans le controller:

```
use Symfony\Contracts\Translation\TranslatorInterface;

public function index(TranslatorInterface $translator): Response
{
    $translated = $translator->trans('Hello there');

    // ...
}
```

Traduction dans un fichier YAML:

```
// translations/messages.fr.yaml

Hello there: Général Kenobi !
```

Rédiger les catalogues de messages

Messages réels ou des mots-clés

On peut gérer la traduction des messages de 2 façons différentes:

- en utilisant les messages rédigés dans la langue des paramètres régionaux par défaut. Ce message est ensuite utilisé comme « identifiant » lors de la création de traductions.
- en utilisant les messages en tant que « mots-clés » qui véhiculent l'idée du message. Le mot-clé message est ensuite utilisé comme « identifiant » pour toutes les traductions.

La méthode des clés est pratique car la clé du message n'aura pas besoin d'être modifiée dans chaque fichier de traduction.

Le choix de la méthode à utiliser dépend entièrement de vous, mais le format « mot clé » est souvent recommandé pour les applications multilingues, tandis que pour les ensembles partagés contenant des ressources de traduction, nous recommandons le vrai message, afin que votre application puisse choisir de désactiver le couche de traduction.

Rédiger les catalogues de messages

Traduire dans le template

La plupart du temps les traductions se font dans les templates. Twig permet de faire les traductions avec le filtre ou le tag `trans`.

filtre

```
{{ message|trans }}
```

```
{{ message|trans({'%name%': 'John'}, 'app') }}
```

tag

```
{% trans with {'%name%': 'John'} from 'app' %}Hello %name%{% endtrans %}
```

```
{% trans with {'%name%': 'John'} from 'app' into 'fr' %}Hello %name%{% endtrans %}
```

Rédiger les catalogues de messages

Domaine de traduction

On peut définir le domaine de traduction pour un template entier avec une seule balise

```
{% trans_default_domain 'app' %}
```

(!) Seul le template actuel sera affecté par le domaine de traduction, pas les templates "inclus"

Traduire les validations, les entités, les urls

Les validations

Les messages associés aux validateurs peuvent être traduit en utilisant le domaine de traduction `validators`.

```
class Author
{
    #[Assert\NotBlank(message: 'author.name.not_blank')]
    public string $name;
}
```

translations/validators/validators.en.yaml

author.name.not_blank: Please enter an author name.

Traduire les validations, les entités, les urls

Les entités

Les champs des entités peuvent être traduits dans les formulaires qui exposent l'entité.

Certaines options des `formTypes` permettent de transmettre une instance de `TranslatableMessage` pour traduire une valeur.

```
$builder
->add('zipCode', options: [
    'label' => new TranslatableMessage('address.zipCode', ['%country%' => $country], 'address'),
])
```

(!) Les form thèmes proposés par Symfony, utilise par défaut le filtre `trans`

Traduire les validations, les entités, les urls

Les urls

Une route peut définir une URL différente pour chaque `locale`.

```
#[Route(path: [  
    'en' => '/about-us',  
    'fr' => '/a-propos'  
], name: 'about')]
```


Les services

Utilisation des services du ServiceContainer

Symfony met à disposition une multitude d'objets utiles: L'objet "Mailer" par exemple pour aider à envoyer des e-mails tandis qu'un autre aide à enregistrer les entités dans la base de données.

Ces objets utiles sont appelés services et chaque service est enregistré dans un objet spécial appelé conteneur de services. Le conteneur de services permet de centraliser la manière dont les objets sont construits.

Le conteneur de service est créé et initialisé au lancement de l'application par le **Kernel**.

Injection de services et de configuration

Dans un contrôleur

Pour faire appel à un service dans un **contrôleur**, on le "demande" au conteneur de service en indiquant le nom de la classe ou de l'interface en argument d'une méthode.

```
use Psr\Log\LoggerInterface;
[...]  
public function index(LoggerInterface $logger): Response  
{  
    $logger->info("Hello service");  
}
```

(!) Il est aussi possible de faire appel à des services dans le **constructeur** du contrôleur

Injection de services et de configuration

Dans un service

Pour faire appel à un service dans un service, il faut créer le constructeur et faire la "demande" en indiquant le nom de la classe ou de l'interface en argument.

```
use Psr\Log\LoggerInterface;

class MessageGenerator
{
    public function __construct(
        private LoggerInterface $logger,
    ) { }

    [...]
}
```

Injection de services et de configuration

Depuis la version 4 de Symfony:

- la configuration par défaut permet d'utiliser la configuration et le cablage automatique des services.
- l'ensemble des classes se trouvant dans le répertoire `src/` sont considérées comme des services. Excepté les classes se trouvant dans les répertoires `src/DependencyInjection/`, `src/Entity/` et le fichier `src/Kernel.php`
- par défaut chaque service est référencé avec son FQCN (Fully-Qualified Class Name) dans le conteneur de service

(!) Pour connaître la liste des services disponible, on utilise la commande `php bin/console debug:autowiring`. On peut aussi ajouter l'option `--all` pour obtenir la liste complète.

Créer un service

Si on souhaite créer du code pour qu'il puisse être réutilisé, il faut alors le créer dans un **service**.

Un **service** est une **classe PHP** créé dans un répertoire du répertoire source (`src/`).

```
namespace App\Service;

class MessageGenerator
{
    public function getHappyMessage(): string
    {
        $messages = [...];

        $index = array_rand($messages);

        return $messages[$index];
    }
}
```

Lorsque vous demandez un service, le conteneur construit un nouvel objet et le renvoie. Mais si vous ne demandez jamais le service, il n'est jamais construit. Les services ne sont créés qu'une seule fois : la même instance est renvoyée à chaque fois que vous en faites la demande.

Les services publics et privés

Par défaut chaque service est privé. Lorsqu'un service est privé, on ne peut pas y accéder directement depuis le conteneur en utilisant `$container->get()`.

Si vous souhaitez rendre un service public, il faut le configurer manuellement:

```
# config/services.yaml
services:
    # ...

    # explicitly configure the service
    App\Service\PublicService:
        public: true
```

(!) Il est recommandé de créer uniquement des services privés et de récupérer les services à l'aide de l'injection de dépendances.

Renommer les services à l'aide des alias

Il est parfois utile de pouvoir accéder à un service en utilisant un raccourcis. Ceci est possible en lui définissant un alias.

```
namespace App\Service;

#[AsAlias(id: 'app.message')]
class MessageGenerator
{
    [...]
}
```

Dans le cas où votre service est public, il devient alors plus facile d'y accéder

```
$container->get('app.message'); // retourne une instance de MessageGenerator
```

L'alias peut aussi être configuré dans le fichier service.yaml

```
services:
    # ...
    app.message: '@App\Service\MessageGenerator'
```


Renommer les services à l'aide des alias

Dans le cas où l'on souhaite utiliser nos services avec des interfaces, il est possible d'utiliser l'attribut `#[Target]`.

Cet attribut permet de câbler automatiquement le service exacte que l'on souhaite injecter en utilisant son alias. Grâce à cela, on peut avoir plusieurs services implémentant la même interface.

```
namespace App\Service;

use Psr\Log\LoggerInterface;

class MessageGenerator
{
    public function __construct(
        #[Target('monolog.logger.request')]
        private LoggerInterface $logger
    ){}

    [...]
}
```

Paramétrer les services et gérer les arguments

Il existe des cas où un argument vers un service ne peut pas être automatiquement câblé. Par exemple, les cas où on souhaite rendre une adresse e-mail (string) configurable.

```
class MyServiceManager
{
    public function __construct(
        private string $name
    ) {
    }

    public function doSomething(): string
    {
        [...]

        return "Hello " . $this->name;
    }
}
```

Paramétrer les services et gérer les arguments

Dans ce genre de cas le conteneur de service ne peut pas déterminer la valeur que l'on souhaite transmettre. Il faut donc configurer le service en lui indiquant explicitement les valeurs des arguments.

```
services:
  ...
  # explicitly configure the service
  App\Service\MyServiceManager:
    arguments:
      $name: 'John Doe'
```

Maintenant le conteneur de service transmettra la valeur "John Doe" à l'argument `$name` du constructeur lors de la création du service.

Paramétrer le câblage auto et la configuration auto

câblage auto

Le fichier `services.yaml` contient un paramètre `autowire: true` dans la section `_defaults` afin qu'il s'applique à tous les services définis dans ce fichier.

```
services:
  # default configuration for services in *this* file
  _defaults:
    autowire: true # Automatically injects dependencies in your services.
```

Avec ce paramètre, vous pouvez saisir des arguments dans la méthode `__construct()` des services et le conteneur transmettra automatiquement les arguments correctement.

Paramétrer le câblage auto et la configuration auto

configuration auto

Le fichier `services.yaml` contient un paramètre `autoconfigure: true` dans la section `_defaults` afin qu'il s'applique à tous les services définis dans ce fichier.

```
services:
  # default configuration for services in *this* file
  _defaults:
    autoconfigure: true # Automatically registers your services as commands, event subscribers, etc.
```

Avec ce paramètre, le conteneur appliquera automatiquement certaines configurations (**tags**) à vos services, en fonction de la classe de votre service. Ceci est utilisé pour marquer automatiquement vos services.

Ex: Toutes les classes qui implémentent l'interface `Twig\Extension\ExtensionInterface`, seront automatiquement taggées avec la valeur `twig.extension`

Lier des arguments par leur nom ou leur type

Dans la section `_default` du fichier `service.yaml`, il existe une propriété `bind` qui permet de lier des arguments spécifiques par leur nom, par leur type ou les deux.

```
services:
  # default configuration for services in *this* file
  _defaults:
    autowire: true      # Automatically injects dependencies in your services.
    autoconfigure: true # Automatically registers your services as commands, event subscribers, etc.
    bind:
      # pass this value to any $name argument for any service
      # that's defined in this file (including controller arguments)
      $name: 'John Doe'

      # pass this service for any LoggerInterface type-hint for any
      # service that's defined in this file
      Psr\Log\LoggerInterface: '@monolog.logger.request'

      # optionally you can define both the name and type of the argument to match
      string $name: 'John Doe'
      Psr\Log\LoggerInterface $requestLogger: '@monolog.logger.request'
```

Bibliographie

- <https://getcomposer.org/>
- <https://symfony.com/download>
- <https://symfony.com/doc/current/index.html>
- https://symfony.com/doc/current/page_creation.html
- <https://symfony.com/doc/current/routing.html>
- <https://symfony.com/doc/current/templates.html>
- <https://symfony.com/doc/current/doctrine.html>
- <https://symfony.com/doc/current/forms.html>
- <https://symfony.com/doc/current/validation.html>
- <https://symfony.com/doc/current/reference/index.html>
- <https://symfony.com/doc/current/configuration.html>
- https://symfony.com/doc/current/service_container/alias_private.html
- https://symfony.com/doc/current/service_container/autowiring.html
- https://symfony.com/doc/current/service_container.html